

性能测试优化规范

第一条 目的与适用范围

为保证数据库大赛性能评测的公平性、可复现性和技术通用性，参赛选手提交的所有性能优化方案均应面向数据库系统的通用处理机制，不得通过识别、匹配或利用特定评测用例的固定特征获取非通用的性能收益。

本规范适用于参赛作品的编译、解析、优化、执行、存储、事务处理、缓存管理、日志处理及其他与性能评测相关的全部环节。

第二条 基本原则

参赛作品所采用的性能优化应同时满足以下要求：

- 通用性原则** 优化方案应适用于具有相同语义或相同执行特征的一类通用场景，不得仅对评测数据、固定SQL、固定表结构或其他特定评测特征生效。
- 语义一致性原则** 优化前后 SQL 的执行结果、事务语义、隔离级别、持久性及错误处理行为应符合赛题规定，不得改变 SQL 原有语义。
- 无额外副作用原则** 不得产生 SQL、事务或系统配置未声明的额外副作用，包括但不限于非法修改数据、跳过必要的持久化流程、隐式降低一致性保障或绕过权限及约束检查。
- 可验证原则** 优化实现应能够通过源代码审查、构建过程检查、运行日志分析、随机用例测试或等价性测试等方式进行验证。

第三条 允许采用的优化

参赛选手可以在不违反本规范其他条款的前提下，采用包括但不限于以下通用优化方式：

- 索引结构、索引选择、索引扫描及索引维护优化；
- SQL 解析、逻辑计划生成、代价估算及物理执行计划选择优化；
- 执行器、算子实现、向量化执行、并行执行及流水线执行优化；
- 连接顺序、连接算法、谓词下推、投影裁剪及常量折叠等通用查询优化；
- 事务管理、锁管理、并发控制及可见性判断优化；
- 数据页、缓冲池、查询结果、中间结果及元数据缓存优化；
- WAL、检查点、刷盘、日志批处理及持久化路径优化；
- 内存分配、对象复用、数据布局、序列化及反序列化优化；
- 通用编译器优化，包括使用GCC编译器的 **-O3** 编译优化选项，但编译器优化不得导致 SQL 结果、事务行为或数据持久性发生不符合赛题要求的变化；
- 其他不依赖特定评测用例、且能够保持系统语义正确性的通用性能优化。

第四条 禁止采用的优化

参赛选手不得针对评测用例编写专用匹配、专用分支或硬编码处理逻辑。禁止行为包括但不限于：

- 根据固定表名、数据库名、索引名、字段名或字段排列顺序触发专用执行逻辑；
- 根据固定字段类型、字段数量、表数量或特定表结构组合识别评测用例；
- 对固定 SQL 文本、SQL 片段、关键字排列、空格格式、注释、常量值或参数值进行匹配；
- 根据特定查询的哈希值、摘要值、长度、编号或其他可识别特征直接返回预设结果；
- 针对已知评测数据内容、数据分布、数据规模或数据生成规律预先计算、缓存或构造结果；

6. 绕过通用 SQL 解析、查询优化、执行器、事务处理、约束检查、WAL 或持久化流程；
7. 在识别出评测环境、评测进程、评测脚本或评测时间后启用仅用于跑分的特殊逻辑；
8. 以其他方式利用评测用例的固定特征获得不具备通用性的性能收益。

上述行为统称为“面向评测用例的硬编码优化”。无论相关逻辑是否直接写入源代码，或通过配置文件、生成代码、预置缓存、外部文件及其他形式实现，均按照相同标准认定。

第五条 审查与违规处理

大赛技术委员会有权通过源代码审查、构建参数检查、运行时跟踪、随机化测试、变形测试及补充用例测试等方式核验参赛作品。

参赛作品存在下列情形之一的，可认定为违规：

1. 存在面向评测用例的硬编码优化；
2. 通过绕过必要流程或改变 SQL 语义提升性能得分；
3. 隐瞒影响评测公平性的特殊构建参数、配置或运行逻辑；
4. 拒绝提供合理的技术说明，或无法证明相关优化具有通用性；
5. 采用其他破坏评测公平性、正确性或可复现性的技术手段。

经确认违规后，大赛组织方有权视情节采取取消相关成绩、取消获奖资格、终止参赛资格或取消后续参赛资格等处理措施。涉嫌违规的参赛队伍应配合赛事组织方进行技术核查和说明。

第六条 解释与补充

对于本规范中未明确列举的优化方式，应遵循通用性、语义一致性、无额外副作用和可验证性等原则，由全国大学生计算机系统能力大赛数据库管理系统设计赛技术委员会进行最终判定。若参赛队伍对当前优化方向的合规性存在疑问，可在竞赛交流群中咨询，也可私下联系助教进行确认。

本规范的最终解释权归**全国大学生计算机系统能力大赛数据库管理系统设计赛技术委员会**所有。技术委员会可根据评测环境和赛事实际情况发布补充规则，补充规则与本规范具有同等效力。

性能测试SQL示例

new_order：读写事务

客户输入一笔新的订单交易

```
select c_discount, c_last, c_credit, w_tax from customer, warehouse where
w_id=:w_id and c_w_id=w_id and c_d_id=:d_id and c_id=:c_id;
select d_next_o_id, d_tax from district where d_id=:d_id and d_w_id=:w_id;
update district set d_next_o_id=:d_next_o_id where d_id=:d_id and d_w_id=:w_id;
insert into orders values(:o_id, :o_d_id, :o_w_id, :o_c_id, :o_entry_d,
:o_carrier_id, :o_ol_cnt, o_all_local);
insert into new_orders values(:o_id, :d_id, :w_id);
select i_price, i_name, i_data from item where i_id=:ol_i_id;
select s_quantity, s_data, s_dist_01, s_dist_02, s_dist_03, s_dist_04, s_dist_05,
s_dist_06, s_dist_07, s_dist_08, s_dist_09, s_dist_10 from stock where
```

```
s_i_id=:ol_i_id and s_w_id = :ol_supply_w_id;
update stock set s_quantity=:s_quantity where s_i_id = :ol_i_id and s_w_id =
:ol_supply_w_id;
insert into order_line values (:ol_o_id, :ol_d_id, :ol_w_id, :ol_number, :ol_i_id,
:ol_supply_w_id, :ol_delivery_d, :ol_quantity, :ol_amount, :ol_dist_info);
```

payment: 读写事务

更新客户账户余额

```
update warehouse set w_ytd=w_ytd+:h_amount where w_id=:w_id;
select w_street_1, w_street_2, w_city, w_state, w_zip, w_name from warehouse where
w_id=:w_id;
update district set d_ytd=d_ytd+:h_amount where d_w_id=:w_id and d_id=:d_id;
select d_street_1, d_street_2, d_city, d_state, d_zip, d_name from district where
d_w_id=:w_id and d_id=:d_id;
select c_first, c_middle, c_last, c_street_1, c_street_2, c_city, c_state, c_zip,
c_phone, c_credit, c_credit_lim, c_discount, c_balance, c_since from customer
where c_w_id=:w_id and c_d_id=:d_id and c_id=:c_id;
update customer set c_balance=:c_balance where c_w_id=:w_id and c_d_id=:d_id and
c_id=:c_id;
insert into history values(:h_c_id, :h_c_d_id, :h_c_w_id, :h_d_id, :h_w_id,
:h_date, :h_amount, :h_data);
```

delivery: 读写事务

发货

```
select min(no_o_id) as min_o_id from new_orders where no_d_id=:d_id and
no_w_id=:w_id;
delete from new_orders where no_o_id=:o_id and no_d_id=:d_id and no_w_id=:w_id;
select o_c_id from orders where o_id=:no_o_id and o_d_id=:d_id and o_w_id=:w_id;
update orders set o_carrier_id=:o_carrier_id where o_id=:no_o_id and o_d_id=:d_id
and o_w_id=:w_id;
update order_line set ol_delivery_d=:datetime where ol_o_id=:no_o_id and
ol_d_id=:d_id and ol_w_id=:w_id;
select sum(ol_amount) as sum_amount from order_line where ol_o_id=:no_o_id and
ol_d_id=:d_id;
update customer set c_balance=:c_balance, c_delivery_cnt=c_delivery_cnt+1 where
c_id=:c_id and c_d_id=:d_id and c_w_id=:w_id;
```

order_status: 只读事务

查询客户最近的交易状态

```
select count(c_id) as count_c_id from customer where c_w_id=:c_w_id and
c_d_id=:c_d_id and c_last=:c_last;
select c_balance, c_first, c_middle, c_last from customer where c_w_id=:c_w_id and
c_d_id=:c_d_id and c_last=:c_last order by c_fisrt;
/* 60%执行上述两条, 40%执行下述一条 */
select c_balance, c_first, c_middle, c_last from customer where c_w_id=:c_w_id and
c_d_id=:c_d_id and c_id=:c_id;
select o_id, o_entry_d, o_carrier_id from orders where o_w_id=:c_w_id and
o_d_id=:c_d_id and o_c_id=:c_id and o_id=:o_id;
select ol_i_id, ol_supply_w_id, ol_quantity, ol_amount, ol_delivery_d from
order_line where ol_w_id=:c_w_id and ol_d_id=:c_d_id and ol_o_id=:o_id;
```

stock-level: 只读事务

查询仓库库存状况

```
select d_next_o_id from district where d_id=:d_id and d_w_id=:w_id;
select ol_i_id from order_line where ol_w_id=:w_id and ol_d_id=:d_id and
ol_o_id<:d_next_o_id and ol_o_id>=:d_next_o_id-20);
select count(*) as count_stock from stock where s_w_id=:w_id and s_i_id=:ol_i_id
and s_quantity<:level;
```